

Лабораторная работа № 6.

CRUD-операции с использованием Prisma

Тема: Реализация полного CRUD Product API через Prisma Client

Цель: Реализовать операции создания, чтения, изменения и удаления товаров через Prisma Client, сохранив существующую архитектуру Express-приложения и REST API.

Дисциплина: Node.js и Express.js

Исходные условия

Продолжайте работу с проектом express-shop. PostgreSQL-база shop_db и Prisma уже должны быть подключены по результатам предыдущей лабораторной работы.

В этой работе необходимо полностью заменить ручные SQL-запросы Product Service операциями Prisma Client.

CRUD endpoints Product API

Метод	URL	Prisma	Назначение
GET	/api/products	findMany()	Получить список товаров
GET	/api/products/:id	findUnique()	Получить один товар
POST	/api/products	create()	Создать товар
PATCH	/api/products/:id	update()	Изменить товар
DELETE	/api/products/:id	delete()	Удалить товар

1. Проверка Prisma

Убедитесь, что Prisma Client сгенерирован после изменений в schema.prisma.

```
PowerShell
npx prisma@7 generate
```

2. Проверка модели Product

В prisma/schema.prisma должна существовать модель Product, связанная с таблицей products.

```
prisma/schema.prisma

model Product {
  id Int @id @default(autoincrement())
  name String @db.VarChar(150)
  price Int
  categoryId Int @map("category_id")
  inStock Boolean @default(true) @map("in_stock")

  category Category @relation(
    fields: [categoryId],
```

```
references: [id]
)

@@map("products")
}
```

Важно

Не создавайте новую базу данных. Prisma должен работать с уже существующей shop_db.

3. Подключение Prisma Client в Product Service

Откройте services/productService.js. В нём не должно быть массива products[] и вызовов pool.query().

```
services/productService.js

import { prisma } from "../config/prisma.js";
```

4. Получение всех товаров

```
getAllProducts()

export async function getAllProducts() {
  return prisma.product.findMany({
    orderBy: {
      id: "asc"
    }
  });
}
```

Метод findMany() возвращает массив товаров. Если таблица пуста, результатом будет пустой массив.

Проверка GET коллекции

Метод	URL	Ожидаемый результат
GET	/api/products	200 и JSON-массив товаров

5. Получение товара по id

```
findProductById()

export async function findProductById(id) {
  return prisma.product.findUnique({
    where: {
      id
    }
  });
}
```

Если товар существует, `findUnique()` возвращает объект. Если записи с таким `id` нет, результат равен `null`.

Проверка поиска по `id`

Метод	URL	Ожидаемый результат
GET	<code>/api/products/1</code>	200 и объект <code>Product</code>
GET	<code>/api/products/9999999</code>	404 <code>Product not found</code>

Архитектура

Prisma не формирует HTTP-статус 404. Service возвращает `null`, а Controller преобразует отсутствие объекта в HTTP-ошибку.

6. Создание товара

```
addProduct()

export async function addProduct(data) {
  return prisma.product.create({
    data: {
      name: data.name,
      price: data.price,
      categoryId: data.categoryId,
      inStock: data.inStock
    }
  });
}
```

Метод `create()` добавляет новую строку в PostgreSQL и возвращает созданный объект с автоматически назначенным `id`.

```
productController.js

export async function createProduct(req, res, next) {
  try {
    const product = await addProduct(req.body);
    res.status(201).json(product);
  } catch (error) {
    next(error);
  }
}
```

POST `/api/products`

```
POST /api/products
Content-Type: application/json

{
  "name": "Gaming Keyboard",
  "price": 28000,
  "categoryId": 1,
  "inStock": true
}
```

```
}
```

Используйте существующий categoryId из таблицы categories.

FOREIGN KEY

Если передать несуществующий categoryId, PostgreSQL отклонит операцию из-за ограничения внешнего ключа.

Пример ответа

```
{
  "id": 21,
  "name": "Gaming Keyboard",
  "price": 28000,
  "categoryId": 1,
  "inStock": true
}
```

Фактическое значение id зависит от состояния вашей базы данных.

7. Проверка CREATE в PostgreSQL

```
SELECT
  id,
  name,
  price,
  category_id,
  in_stock
FROM products
ORDER BY id;
```

Новая строка должна существовать непосредственно в таблице products.

8. Изменение товара

Для сохранения прежнего поведения API сначала проверьте существование Product через findUnique(), а затем выполните update().

```
updateProductById()

export async function updateProductById(id, changes) {
  const existing = await prisma.product.findUnique({
    where: {
      id
    }
  });

  if (!existing) {
    return null;
  }

  return prisma.product.update({
    where: {
      id
    },
```

```
data: changes
});
}
```

Validation

Объект changes должен содержать только разрешённые поля Product. Проверяйте и фильтруйте PATCH-запрос до вызова Service.

```
productController.js

export async function updateProduct(req, res, next) {
  try {
    const id = Number(req.params.id);
    const product = await updateProductById(id, req.body);

    if (!product) {
      return next(new AppError("Product not found", 404));
    }

    res.json(product);
  } catch (error) {
    next(error);
  }
}
```

```
PATCH /api/products/:id

PATCH /api/products/21
Content-Type: application/json

{
  "price": 25000
}
```

Используйте фактический id товара. После PATCH должна измениться только цена, остальные свойства должны сохраниться.

9. Проверка UPDATE в PostgreSQL

```
SELECT
  id,
  name,
  price,
  category_id,
  in_stock
FROM products
WHERE id = 21;
```

Используйте фактический id. Значение price должно соответствовать PATCH-запросу.

10. Удаление товара

```
deleteProductById()

export async function deleteProductById(id) {
  const existing = await prisma.product.findUnique({
    where: {
      id
    }
  });

  if (!existing) {
    return null;
  }

  return prisma.product.delete({
    where: {
      id
    }
  });
}
```

```
productController.js

export async function deleteProduct(req, res, next) {
  try {
    const id = Number(req.params.id);
    const product = await deleteProductById(id);

    if (!product) {
      return next(new AppError("Product not found", 404));
    }

    res.json(product);
  } catch (error) {
    next(error);
  }
}
```

```
DELETE /api/products/:id

DELETE /api/products/21
```

При успешном удалении API должен вернуть объект удалённого Product.

11. Проверка удаления

```
GET /api/products/21
```

После удаления запрос должен вернуть 404 Product not found.

```
SELECT *
FROM products
WHERE id = 21;
```

SQL-запрос должен вернуть 0 строк. Используйте фактический id удалённого товара.

12. Фильтрация списка

Проверьте работу `findMany()` с фильтрацией и сортировкой.

Пример `findMany()`

```
const products = await prisma.product.findMany({
  where: {
    price: {
      gte: 10000
    },
    inStock: true
  },
  orderBy: {
    price: "asc"
  }
});
```

`where` задаёт условия отбора, а `orderBy` задаёт сортировку результата.

13. Проверка через Prisma Studio

PowerShell

```
npx prisma@7 studio
```

Используйте Prisma Studio для дополнительной проверки результатов CREATE, UPDATE и DELETE.

Проверка

Prisma Studio не заменяет тестирование REST API. Основная проверка выполняется через HTTP-запросы.

14. Полный CRUD-цикл

1. Создайте новый Product через POST `/api/products`.
2. Получите созданный Product через GET `/api/products/:id`.
3. Измените только цену через PATCH `/api/products/:id`.
4. Снова получите Product и убедитесь, что цена изменилась.
5. Удалите Product через DELETE `/api/products/:id`.
6. Повторите GET и убедитесь, что API возвращает 404.

15. Итоговый Product Service

`services/productService.js`

```
import { prisma } from "../config/prisma.js";

export async function getAllProducts() {
  return prisma.product.findMany({
    orderBy: {
      id: "asc"
    }
  });
}
```

```

    });
  }

export async function findProductById(id) {
  return prisma.product.findUnique({
    where: {
      id
    }
  });
}

export async function addProduct(data) {
  return prisma.product.create({
    data: {
      name: data.name,
      price: data.price,
      categoryId: data.categoryId,
      inStock: data.inStock
    }
  });
}

export async function updateProductById(id, changes) {
  const existing = await prisma.product.findUnique({
    where: {
      id
    }
  });

  if (!existing) {
    return null;
  }

  return prisma.product.update({
    where: {
      id
    },
    data: changes
  });
}

export async function deleteProductById(id) {
  const existing = await prisma.product.findUnique({
    where: {
      id
    }
  });

  if (!existing) {
    return null;
  }

  return prisma.product.delete({
    where: {
      id
    }
  });
}

```

Требования к результату

- В Product Service отсутствует pool.query().
- GET /api/products использует findMany().
- GET /api/products/:id использует findUnique().
- POST /api/products использует create().
- PATCH /api/products/:id использует update().
- DELETE /api/products/:id использует delete().
- POST возвращает статус 201.
- Несуществующий Product приводит к 404.
- Validation middleware продолжает использоваться.
- Routes не содержат Prisma-запросов.
- Controllers не содержат Prisma-запросов.
- Все изменения сохраняются в PostgreSQL.

Критерии оценивания

№	Критерий
1	Prisma Client корректно подключён к проекту
2	findMany() возвращает список товаров
3	findUnique() возвращает товар по id
4	Отсутствующий товар обрабатывается как 404
5	create() создаёт Product
6	POST возвращает 201 Created
7	update() изменяет переданные поля
8	PATCH не перезаписывает остальные поля
9	delete() удаляет Product
10	После DELETE повторный GET возвращает 404
11	Product Service не содержит ручного SQL
12	Controller не содержит Prisma-запросов
13	Route не содержит Prisma-запросов
14	Validation продолжает работать
15	Студент может объяснить соответствие CRUD и методов Prisma